EncryptEdge Labs

Junior Application Security Engineer Internship Programme

Junior Application Security Engineer

Internship Task Report

Intern Full Name	Abdoul Karim Savadogo
Intern Email	abdoulkarimsavadogo03@gmail.com
Cohort	2026
Task ID	ASE-APPSEC
Task Title	Application Security Assessment of OWASP Juice Shop (OWASP Top 10, DAST with ZAP, Threat Modelling)
Skill Domain	Web Application Security / DAST / Threat Modelling
Difficulty Level	Beginner
CPE Credits	Per task brief
Guiding Framework(s)	OWASP Top 10 (2021 / 2025), CWE, 5×5 likelihood × impact risk matrix
Submission Date	12 October 2026

Copyright © 2026 EncryptEdge Labs Limited | Company No. 15830711 | United Kingdom
All activities in this report were performed within authorised lab environments as defined by the task brief.

Executive Summary

Purpose and scope. This report documents a structured application-security assessment of OWASP Juice Shop, a deliberately vulnerable web application, deployed as a Docker container in an isolated Kali Linux lab and reached at http://localhost:3000. The work covered the OWASP Top 10 (Phase A), lab setup (Phase B), a dynamic scan with OWASP ZAP 2.17.0 (Phase C), threat modelling (Phase D), hosted TryHackMe labs (Phase E) and this report (Phase F).

Overall result. Ten findings were recorded: three confirmed by the ZAP scan and seven drawn from the threat model. By severity: **6 Critical, 1 High, 2 Medium, 1 Low**. The ZAP scan raised no High-risk alerts. Its confirmed issues are configuration weaknesses: no Content-Security-Policy header and a wildcard CORS policy (both Medium), and a disclosed Unix timestamp (Low). The most serious risks come from the threat model: SQL injection (score 25), then cross-site scripting, broken authentication, broken access control and sensitive data exposure (20 each) and CSRF (16). These were not exploited in this task and the scan did not exercise them, because every endpoint it covered was requested with GET.

What matters most. The confirmed Medium findings are inexpensive to fix and matter beyond their rating: without a CSP, any cross-site scripting flaw has no second line of defence, and user data and authentication tokens are the assets most exposed. For a production system, the first priorities would be:

- **Fix the Critical threats first:** parameterised queries (SQL injection), server-side authorisation and ownership checks, strong JWT signing and removal of default credentials, and output encoding with a strict CSP (Section 6.1).
- **Close the configuration gaps:** add a Content-Security-Policy header and replace `Access-Control-Allow-Origin: *` with an explicit allow-list.
- **Build detection and testing into delivery:** an automated ZAP baseline and security-header check in CI/CD, plus logging and alerting on failed logins and access-control failures (Section 6.2).

Limitations. Findings 4–10 are modelled risks, not demonstrated exploits. Phase D scored them by likelihood and impact. The ZAP report covers one target and a limited scan: passive alerts plus one active rule, with GET requests only. A hands-on test of the Critical threats is the logical next step.

1.0 Task Overview

Task ID	ASE-APPSEC
Task Title	Application Security Assessment of OWASP Juice Shop (OWASP Top 10, DAST with ZAP, Threat Modelling)
Skill Domain	Web Application Security / DAST / Threat Modelling
Difficulty	Beginner
Duration	Approx. 9 hours (sum of the six phase estimates)
CPE Credits	Per task brief
Tools Used	Kali Linux, Docker, OWASP Juice Shop (bkimminich/juice-shop), OWASP ZAP 2.17.0 (by Checkmarx), web browser, TryHackMe, Claude (report structuring)
Target Platform	OWASP Juice Shop — Docker container on Kali Linux at http://localhost:3000; TryHackMe hosted rooms
Guiding Framework(s)	OWASP Top 10, CWE, 5×5 likelihood × impact risk matrix
Regulatory Alignment	OWASP Top 10 category mappings (2021 and 2025, as tagged by ZAP) and CWE identifiers. No other standards were specified for this task.

1.1 Introduction

This task assesses the security of OWASP Juice Shop, a deliberately vulnerable web application, using a structured application-security workflow: study the OWASP Top 10, deploy an isolated target, scan it with OWASP ZAP, threat-model its assets and complete hosted labs. It matters because an Application Security Engineer has to turn scattered tool output into a prioritised, evidence-based view of risk that developers and business owners can act on. This report consolidates Phases A–E into findings, a risk assessment and remediation guidance.

1.2 Objectives & Learning Outcomes

The objectives of the task and how each was met:

- **Understand the OWASP Top 10 and relate it to real incidents** — Phase A summarises all ten categories and analyses breach case studies (Facebook 2019, Capital One 2019, Equifax 2017, LinkedIn 2012, TalkTalk 2015).
- **Prepare an isolated, reproducible target** — Phase B deployed Juice Shop as a Docker container on Kali Linux and confirmed it was running on port 3000.
- **Perform dynamic analysis and interpret the results** — Phase C scanned http://localhost:3000 with ZAP 2.17.0: 5 alert types (2 Medium, 1 Low, 2 Informational), analysed in Section 4.
- **Threat-model the application and prioritise risk** — Phase D identified 6 assets and 12 threats, scored them on a 5×5 matrix and proposed mitigations for the high-risk items (Section 5).
- **Apply the knowledge in hosted labs** — Phase E: six TryHackMe rooms completed (Figure 5).
- **Compile a professional report** — Phase F: this document, with an executive summary, findings, risk matrix, mitigations and recommendations.

1.3 Scope & Legal Authorisation

All activities in this task were performed within authorised, isolated environments set up for it. No external systems, production networks or third-party infrastructure were targeted.

Authorised environments used:

- Kali Linux virtual machine (local) — analyst workstation hosting Docker and ZAP
- OWASP Juice Shop — Docker container (bkimminich/juice-shop) at http://localhost:3000
- TryHackMe — six hosted rooms listed in Phase E

Confirmation: no external systems, production networks or third-party infrastructure were accessed. The only non-local host in the ZAP data is https://ads.mozilla.org (one POST request, no alerts raised), which appears to be background browser traffic passing through the ZAP proxy; it was not a test target. AI assistance was limited to structuring and wording this report and is declared in Section 2.3.

2.0 Environment & Tool Setup

2.1 Lab Architecture

The lab was a single Kali Linux workstation running Docker, with Juice Shop as the target and ZAP as the scanner, so target and scanner shared one host (localhost).

Component / VM	OS / Version	Role in Lab	Notes
Kali Linux VM	Kali Linux	Analyst workstation; hosts Docker and ZAP	Prompt kali@kali; Docker commands run with sudo
Docker Engine	Docker on Kali host	Container runtime	`sudo docker run -d -p 3000:3000 bkimminich/juice-shop`
Juice Shop container	bkimminich/juice-shop (Node.js)	Target application	ID 91a7ddf53486, name musing_stonebraker, 0.0.0.0:3000→3000/tcp
OWASP ZAP	2.17.0 (by Checkmarx)	DAST scanner and proxy	Target http://localhost:3000; report 28 Sep 2026
TryHackMe	Hosted platform	Hosted labs (Phase E)	Profile abdoulkarimsavad



Figure 1 — Juice Shop started on Kali Linux: `docker ps` is empty, `docker run` starts the container, and `docker ps` then shows it Up with port 3000 mapped.

2.2 Tool Configuration

The two tools that needed configuration are described below.

Tool Name	Docker Engine and the OWASP Juice Shop image (bkimminich/juice-shop)
Installation Method	Docker Engine on Kali Linux; Juice Shop image from Docker Hub
Configuration Steps	Container started detached with host port 3000 published to container port 3000: `sudo docker run -d -p 3000:3000 bkimminich/juice-shop`. No other settings were changed.
Verification	`sudo docker ps` showed the container with status “Up 2 seconds” and ports 0.0.0.0:3000→3000/tcp (Figure 1).

Tool Name	OWASP ZAP 2.17.0 (by Checkmarx)
Installation Method	Installed on the Kali Linux workstation
Configuration Steps	Target http://localhost:3000. The alerts raised came from passive scanners plus one active rule (User Agent Fuzzer). The HTML report was exported with all contexts, risk levels High to Informational and all confidence levels.
Verification	Report generated 28 Sep 2026, 19:39:03, listing 5 alert types for http://localhost:3000 (Figures 2 and 3).

2.3 AI Tool Usage Declaration

Claude (Anthropic) was used for Phase F, as the task brief allows. It was not used to produce the lab work or evidence.

AI Tool	Used For	What Was NOT Done with AI
Claude (Anthropic)	Structuring the report sections (Executive Summary, Findings, Recommendations), consolidating the Phase A–E evidence into this template, and drafting and refining the report wording.	AI did not deploy the lab, run the ZAP scan, take the screenshots or produce the Phase A–E evidence files. The findings come from the ZAP report and the threat model produced during the task.

The scan output, screenshots and threat model are the intern’s own work; Claude assisted with structure and wording as declared above, and the intern has reviewed the report.

3.0 Methodology — Phase Execution

This section records what was done in each phase and what it showed.

Phase A — Theory & Conceptual Foundation

Phase Objective	Build a conceptual foundation in the OWASP Top 10 by summarising each category and linking it to a real-world incident.
Estimated Time	~60 minutes
Evidence File(s)	PhaseA_OWASP_Top10_Summary.md

Actions Taken

I reviewed the ten OWASP Top 10 categories and wrote a short summary of each covering what the weakness is, how it arises and how to prevent it: Broken Access Control, Security Misconfiguration, Software Supply Chain Failures, Cryptographic Failures, Injection, Insecure Design, Authentication Failures, Software or Data Integrity Failures, Security Logging and Alerting Failures, and Mishandling of Exceptional Conditions.

For five categories I researched a real breach: the Facebook contact importer (2019, Broken Access Control), Capital One (2019, Security Misconfiguration), Equifax (2017, Supply Chain), LinkedIn (2012, Cryptographic Failures) and TalkTalk (2015, Injection).

Key Observations

Each breach combined a technical weakness with an operational failure: an unpatched Apache Struts and an expired monitoring certificate at Equifax, an over-privileged WAF role and IMDSv1 at Capital One, unsalted SHA-1 password hashes at LinkedIn, unpatched legacy pages and missed earlier attacks at TalkTalk, and missing rate limits and authorisation checks on bulk lookups at Facebook.

Several categories (Insecure Design, Supply Chain, Logging and Alerting) are about process rather than code, which is why threat modelling, patching and monitoring recur in the later phases.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseA_OWASP_Top10_Summary.md — summary of all ten categories, with five real-world case studies

Phase B — Lab Setup & Environment Preparation

Phase Objective	Deploy an isolated, intentionally vulnerable target (OWASP Juice Shop) on the Kali Linux lab machine.
Estimated Time	~90 minutes
Evidence File(s)	PhaseB_Docker_JuiceShop.png (Figure 1)

Actions Taken

On Kali Linux I ran `sudo docker ps` to confirm that no containers were running, then started Juice Shop with `sudo docker run -d -p 3000:3000 bkimminich/juice-shop`. Docker returned the container ID (91a7ddf53486…), and a second `sudo docker ps` showed the container, named musing_stonebraker, with status “Up 2 seconds” and port 3000 mapped to the host (Figure 1).

Key Observations

The container was published on 0.0.0.0:3000 (and [::]:3000), which exposes it on all host interfaces. For a deliberately vulnerable application, binding to loopback with `-p 127.0.0.1:3000:3000` would keep it reachable only from the lab machine. The target was reachable at http://localhost:3000 and served as the scan target in Phase C.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseB_Docker_JuiceShop.png — terminal screenshot of the container start-up (Figure 1)

Phase C — Core Technical Execution

Phase Objective	Run a dynamic scan against the target and analyse the results.
Estimated Time	~120 minutes
Evidence File(s)	PhaseC_ZAP_Report.html (Figures 2 and 3)

Actions Taken

I scanned http://localhost:3000 with ZAP 2.17.0 and exported the HTML report (generated 28 Sep 2026, 19:39:03) with all risk and confidence levels included. ZAP saw 67 endpoints on the target, all requested with GET, and raised 5 alert types with 5 instances each: 2 Medium, 1 Low and 2 Informational, and none High (Figures 2 and 3).

Key Observations

The scan surfaced configuration and information-disclosure weaknesses, not injection or authentication flaws. That reflects the scan rather than the application: the alerts came from passive scanners plus one active rule, and every request was a GET, so input-driven attacks such as SQL injection were not exercised. Juice Shop is deliberately vulnerable, and the threats scored in Phase D remain unverified by this scan.

The two Medium alerts are tagged SYSTEMIC by ZAP, meaning they apply across the application rather than to a single page.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseC_ZAP_Report.html — full ZAP scanning report
- Figure 2 — alert counts by risk, confidence and type; Figure 3 — alert list

Detailed Technical Walkthrough

The scan details, outputs, decisions and anomalies behind the Phase C results:

- **Configuration:** target http://localhost:3000 (the Phase B container); ZAP 2.17.0; report parameters were all contexts, risk levels High to Informational, and all confidence levels.
- **Outputs:** alerts by risk were High 0, Medium 2, Low 1, Informational 2 (Figure 2). By type: Content Security Policy (CSP) Header Not Set (Medium, High confidence); Cross-Domain Misconfiguration (Medium, Medium confidence); Timestamp Disclosure – Unix (Low, Low confidence); Modern Web Application and User Agent Fuzzer (Informational) (Figure 3).
- **Decisions:** the two Informational alerts describe the application rather than a weakness, so they are not listed as findings. Modern Web Application only indicates a JavaScript single-page app (the standard spider found no links, so ZAP suggests its Client Spider), and User Agent Fuzzer found no difference to

report. The Unix timestamp is kept as a Low finding: 1666666667 decodes to 24 Oct 2022 22:57:47 UTC and, being a round number, looks like a constant in the page rather than server time.

- **Anomalies:** ZAP also recorded https://ads.mozilla.org (one POST endpoint), which is background browser traffic and not a test target; no alerts were raised for it. ZAP's insights show 91% memory use, 2 logged errors, 11 warnings and 10% slow responses on the target. The report still completed, but a memory-constrained scan may be less thorough. The sample response for /sitemap.xml was the application's HTML front page, consistent with a single-page app serving index.html for unknown routes.

Alert Counts by Risk and Confidence

This table shows the number of alerts for each level of risk and confidence included in the report.

(The percentages in brackets represent the count as a percentage of the total number of alerts included in the report, rounded to one decimal place.)

		Confidence				
		User Confirmed	High	Medium	Low	Total
Risk	High	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)	0 (0.0%)
	Medium	0 (0.0%)	1 (20.0%)	1 (20.0%)	0 (0.0%)	2 (40.0%)
	Low	0 (0.0%)	0 (0.0%)	0 (0.0%)	1 (20.0%)	1 (20.0%)
	Informational	0 (0.0%)	0 (0.0%)	2 (40.0%)	0 (0.0%)	2 (40.0%)
	Total	0 (0.0%)	1 (20.0%)	3 (60.0%)	1 (20.0%)	5 (100%)

Alert Counts by Alert Type

This table shows the number of alerts of each alert type, together with the alert type's risk level.

(The percentages in brackets represent each count as a percentage, rounded to one decimal place, of the total number of alerts included in this report.)

Alert type	Risk	Count
Content Security Policy (CSP) Header Not Set	Medium	5 (100.0%)
Cross-Domain Misconfiguration	Medium	5 (100.0%)
Timestamp Disclosure - Unix	Low	5 (100.0%)
Modern Web Application	Informational	5 (100.0%)
User Agent Fuzzer	Informational	5 (100.0%)
Total		5

Figure 2 — ZAP report summary: alert counts by risk and confidence (top) and by alert type (bottom). No High alerts; 2 Medium, 1 Low, 2 Informational.

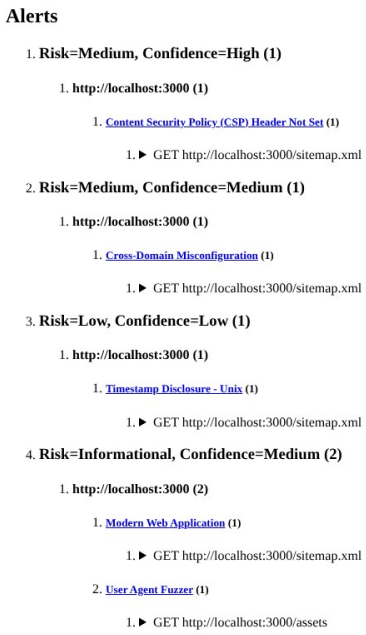


Figure 3 — ZAP alert list for http://localhost:3000, grouped by risk and confidence.

Phase D — Independent Extension & Threat Modelling

Phase Objective	Threat-model OWASP Juice Shop: identify assets and threats, build a risk matrix and propose mitigations for the high-risk items.
Estimated Time	~90–120 minutes
Evidence File(s)	PhaseD_ThreatModel.md (Figure 4)

Actions Taken

I followed a four-step method: (1) identified six key assets: user data (A1), payment information (A2), authentication tokens (A3), admin functions (A4), application files and configuration (A5) and service availability (A6); (2) listed twelve threats, from SQL injection (T1) to insufficient logging and monitoring (T12); (3) scored each threat on a 5×5 likelihood × impact matrix (Figure 4); and (4) proposed mitigations for the high-risk items.

Key Observations

Six threats reach the Critical band (15–25): SQL injection (25), then broken authentication, cross-site scripting, broken access control and sensitive data exposure (20 each), and CSRF (16). Brute force (12) is High, and five threats score 6–9 (Medium). The top threats converge on the same assets, user data and authentication tokens, so one weakness such as XSS stealing the JWT cookie can chain into account takeover.

The Phase C finding “CSP header not set” links directly to T3: without a CSP an XSS flaw has no second line of defence.

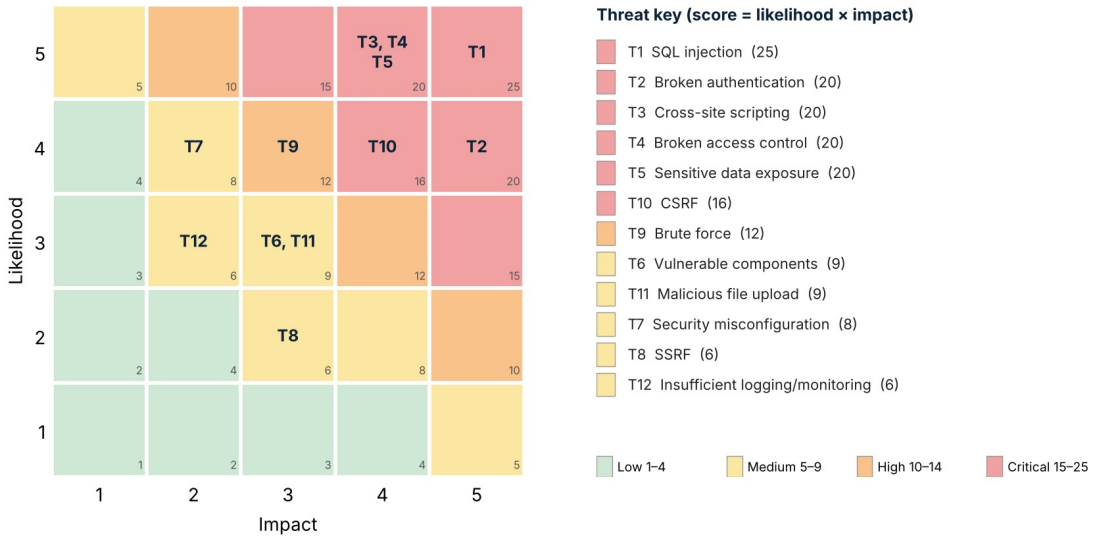


Figure 4 — Risk heat-map from the Phase D threat model: likelihood × impact for T1–T12 (score in each cell's corner).

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseD_ThreatModel.md — assets, threat list, risk matrix and mitigation table
- Figure 4 — risk heat-map of the twelve threats

Phase E — Hosted Lab Integration

Phase Objective	Consolidate and apply web security knowledge through hosted TryHackMe labs.
Estimated Time	~90–120 minutes
Evidence File(s)	PhaseE_TryHackMe_Profile.png (Figure 5)

Actions Taken

I completed six TryHackMe rooms, shown on the profile's Completed rooms tab (Figure 5): Pentesting Fundamentals, Offensive Security Intro, Defensive Security Intro, Careers in Cyber, Web Application Basics and Web Security Essentials. Five are rated Easy; Careers in Cyber is an information room.

Key Observations

The rooms cover the ground used in this project: pentest ethics and methodology, how web applications work (HTTP, URLs, request methods, response codes and headers) and common web risks. That vocabulary underpinned the header and CORS findings in Phase C and the OWASP-style threats in Phase D. The profile shows 6 completed rooms and a 2-day streak.

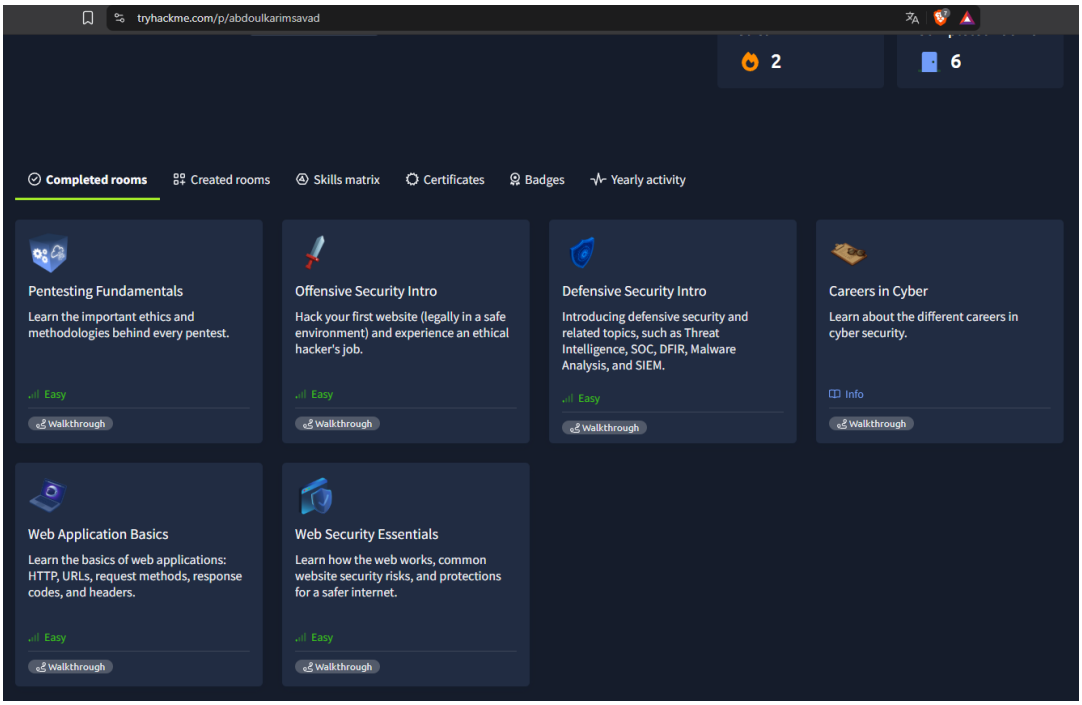


Figure 5 — TryHackMe profile (abdoulkarimsavad): six completed rooms.

Evidence Produced

Evidence files and artefacts for this phase:

- PhaseE_TryHackMe_Profile.png — TryHackMe profile, Completed rooms tab (Figure 5)

Phase F — Documentation & Reporting

Phase Objective	Compile all findings into a professional report.
Estimated Time	~90 minutes
Evidence File(s)	ASE_APPSEC_Report.pdf

Actions Taken

I consolidated the Phase A–E evidence (the OWASP summary, lab screenshot, ZAP report, threat model and TryHackMe profile) into the EncryptEdge Labs report template. Claude helped structure the Executive Summary, Findings and Recommendations sections and refine the wording, as declared in Section 2.3. I then embedded the screenshots and risk matrix, added mitigation strategies and exported the report as ASE_APPSEC_Report.pdf.

Key Observations

The ZAP findings and the threat model use different scales, so the three scanner findings were given likelihood and impact scores in Section 5.1 to fit the same matrix. The template's score guide (15–25 = Critical) puts CSRF (score 16) in the Critical band, whereas the Phase D sheet labelled it High; this report uses the template scale.

Evidence Produced

Evidence files and artefacts for this phase:

- ASE_APPSEC_Report.pdf — this report

4.0 Findings

Ten findings are recorded. Findings 1–3 were confirmed by the Phase C ZAP scan against the live Juice Shop instance. Findings 4–10 come from the Phase D threat model; they are rated by likelihood and impact and were not exploited in this task. The two Informational ZAP alerts (Modern Web Application, User Agent Fuzzer) are described in Phase C and are not listed as findings.

4.1 Findings Summary

#	Finding Title	Type / Category	Framework Ref.*	Phase Identified	Severity / Impact	Evidence Ref.
1	Content Security Policy (CSP) header not set	Security Misconfiguration (confirmed, ZAP)	A02:2025 / CWE-693	Phase C	Medium	Fig. 2–3
2	Permissive CORS policy (wildcard origin)	Broken Access Control / misconfiguration (confirmed, ZAP)	A01:2025 / CWE-264	Phase C	Medium	Fig. 2–3
3	Unix timestamp disclosure	Information disclosure (confirmed, ZAP)	A01:2025 / CWE-497	Phase C	Low	Fig. 2–3
4	SQL injection (T1)	Injection (threat model)	A05:2025 / CWE-89	Phase D	Critical	Fig. 4
5	Cross-site scripting (T3)	Injection (threat model)	A05:2025 / CWE-79	Phase D	Critical	Fig. 4
6	Broken authentication (T2)	Authentication failure (threat model)	A07:2025 / CWE-287	Phase D	Critical	Fig. 4
7	Broken access control (T4)	Broken Access Control (threat model)	A01:2025 / CWE-284	Phase D	Critical	Fig. 4
8	Sensitive data exposure (T5)	Cryptographic failure / misconfiguration (threat model)	A04:2025, A02:2025 / CWE-200	Phase D	Critical	Fig. 4
9	Cross-site request forgery (T10)	Broken Access Control (threat model)	A01:2025 / CWE-352	Phase D	Critical (score 16)	Fig. 4
10	Brute force (T9)	Authentication failure (threat model)	A07:2025 / CWE-307	Phase D	High	Fig. 4

* Framework Ref: OWASP Top 10:2025 category numbers (as used in Phase A and in ZAP's tags) and CWE identifiers.

4.2 Detailed Finding Analysis

Detailed analysis of the five findings that matter most for this assessment: the three confirmed scanner findings, plus SQL injection and cross-site scripting. Findings 6–10 are covered in Sections 5 and 6.

Finding 1: Content Security Policy (CSP) header not set

Finding Title	Content Security Policy (CSP) header not set
Type / Category	Security Misconfiguration (confirmed by ZAP passive scan)
Description	The application does not send a Content-Security-Policy header. ZAP raised the alert on 5 instances and tagged it SYSTEMIC; the example URL is GET http://localhost:3000/sitemap.xml. Without a CSP the browser has no policy limiting which scripts, frames or other resources a page may load.
How Identified	ZAP 2.17.0 passive rule “Content Security Policy (CSP) Header Not Set” during the Phase C scan of http://localhost:3000 (Figures 2 and 3).
Framework Reference	OWASP A02:2025 Security Misconfiguration (A05:2021) / CWE-693 / WASC-15
Business / Operational Impact	Removes a key defence-in-depth control against cross-site scripting and data injection. If an XSS flaw exists (Finding 5), nothing stops injected script from loading and running, which puts user data and authentication tokens at risk (confidentiality, integrity).
Severity / Impact	Medium (ZAP confidence: High)
Evidence Reference	Figures 2 and 3; PhaseC_ZAP_Report.html

Finding 2: Permissive CORS policy (wildcard origin)

Finding Title	Permissive CORS policy (wildcard origin)
Type / Category	Broken Access Control / misconfiguration (confirmed by ZAP passive scan)
Description	Responses carry the header `Access-Control-Allow-Origin: *`, so scripts from any origin may read them. ZAP raised the alert on 5 instances (example: GET http://localhost:3000/sitemap.xml).
How Identified	ZAP passive rule “Cross-Domain Misconfiguration”; the evidence field shows the wildcard header in the HTTP response (Figures 2 and 3).
Framework Reference	OWASP A01:2025 Broken Access Control (A01:2021) / CWE-264 / WASC-14
Business / Operational Impact	Any third-party site can make a visitor’s browser read data returned by unauthenticated endpoints. ZAP notes that browsers do not share authenticated (credentialed) responses under a wildcard, which limits the risk, but data protected only by network position or exposed on unauthenticated APIs could be read by an attacker’s page (confidentiality).
Severity / Impact	Medium (ZAP confidence: Medium)
Evidence Reference	Figures 2 and 3; PhaseC_ZAP_Report.html

Finding 3: Unix timestamp disclosure

Finding Title	Unix timestamp disclosure
Type / Category	Information disclosure (confirmed by ZAP passive scan)
Description	A Unix timestamp, 1666666667 (24 Oct 2022 22:57:47 UTC), appears in responses (5 instances). The round value suggests a constant in the application rather than live server time.
How Identified	ZAP passive rule “Timestamp Disclosure – Unix”; the evidence field shows 1666666667 (Figures 2 and 3).
Framework Reference	OWASP A01:2025 per ZAP’s tags (A01:2021) / CWE-497 / WASC-13
Business / Operational Impact	Timestamps can help an attacker fingerprint systems or predict time-based values. Here the practical impact is minimal unless the value proves sensitive; ZAP recommends confirming this manually (confidentiality, low).
Severity / Impact	Low (ZAP confidence: Low)
Evidence Reference	Figures 2 and 3; PhaseC_ZAP_Report.html

Finding 4: SQL injection (T1)

Finding Title	SQL injection (T1)
Type / Category	Injection (threat model; not exploited in this task)
Description	SQL injection was the highest-scoring threat. Juice Shop stores data in an SQLite database (asset A6), and any endpoint that builds queries from user input could let an attacker change a query to read or modify data, including user records and password hashes (A1, A3). The Phase C scan used GET requests only and raised no SQL injection alert, so this threat is modelled, not confirmed.
How Identified	Phase D threat modelling: asset and threat identification and the 5×5 risk matrix (Figure 4).
Framework Reference	OWASP A05:2025 Injection (A03:2021) / CWE-89
Business / Operational Impact	Possible disclosure of all stored user data and credentials, tampering with accounts or orders, and loss of availability. Phase A’s TalkTalk case (2015) shows the scale: SQLMap against legacy pages exposed the records of 156,959 customers.
Severity / Impact	Critical (likelihood 5 × impact 5 = 25)
Evidence Reference	Figure 4; PhaseD_ThreatModel.md

Finding 5: Cross-site scripting (T3)

Finding Title	Cross-site scripting (T3)
Type / Category	Injection (threat model; not exploited in this task)
Description	XSS was scored Critical. The authentication token is a JWT kept in a cookie (asset A3); if that cookie is readable by script, injected JavaScript could steal it or act as the user. The Phase C scan did not test for XSS, so this is a modelled threat.
How Identified	Phase D threat modelling (Figure 4), read together with the Phase C CSP finding (Finding 1).
Framework Reference	OWASP A05:2025 Injection (A03:2021) / CWE-79
Business / Operational Impact	Session or token theft, account takeover, defacement and malicious redirects (confidentiality, integrity). Finding 1 amplifies the impact because no CSP limits what injected script can do.
Severity / Impact	Critical (likelihood 5 × impact 4 = 20)
Evidence Reference	Figure 4; PhaseD_ThreatModel.md

5.0 Risk Assessment & Threat Model

This section maps the findings to business assets and scores them, then sets out where defensive controls can detect or prevent each one and what happens if the risks stay open. Likelihood and impact for Findings 4–10 come from the Phase D threat model (Figure 4); Findings 1–3 were not scored in Phase D, so their scores are assessed here from the ZAP rating and context.

5.1 Risk Matrix

Risk score = likelihood × impact (each 1–5). Business assets are the Phase D asset IDs: A1 user data, A2 payment information, A3 authentication tokens, A4 admin functions, A5 application files and configuration, A6 service availability.

Finding / Risk	Business Asset	Framework Ref.	Likelihood 1–5	Impact 1–5	Score	Rating
1. CSP header not set	A1, A3 — user data and tokens	A02:2025 / CWE-693	3	3	9	Medium
2. Permissive CORS policy	A1, A6 — API responses	A01:2025 / CWE-264	3	3	9	Medium
3. Unix timestamp disclosure	A5 — application files and config	A01:2025 / CWE-497	2	1	2	Low
4. SQL injection (T1)	A1, A3, A6 — user data, tokens, database	A05:2025 / CWE-89	5	5	25	Critical
5. Cross-site scripting (T3)	A1, A3 — user data, JWT cookie	A05:2025 / CWE-79	5	4	20	Critical
6. Broken authentication (T2)	A3, A4 — tokens, admin functions	A07:2025 / CWE-287	4	5	20	Critical
7. Broken access control (T4)	A1, A4 — user data, admin functions	A01:2025 / CWE-284	5	4	20	Critical
8. Sensitive data exposure (T5)	A1, A5 — user data, files and backups	A04:2025 / CWE-200	5	4	20	Critical
9. CSRF (T10)	A1, A2 — user data, payments	A01:2025 / CWE-352	4	4	16	Critical
10. Brute force (T9)	A3 — authentication tokens	A07:2025 / CWE-307	4	3	12	High

Score guide: 1–4 = Low | 5–9 = Medium | 10–14 = High | 15–25 = Critical

Phase D also scored five threats in the Medium band: vulnerable components (T6, 9), malicious file upload (T11, 9), security misconfiguration (T7, 8), SSRF (T8, 6) and insufficient logging and monitoring (T12, 6). They are shown in Figure 4 and addressed by the strategic recommendations in Section 6.2. CSRF (T10, score 16) was labelled High in Phase D but is Critical on this scale.

5.2 Defensive Opportunities & Coverage Gaps

Where a control could prevent, detect or reduce each finding:

Finding / Gap	Observable / Trigger	Recommended Control	Tool / Implementation
1. CSP header not set	Responses without a Content-Security-Policy header; CSP violation reports once enabled	Send a restrictive CSP application-wide; trial it in Report-Only mode first	Express helmet or reverse-proxy header config; ZAP baseline and header check in CI
2. Permissive CORS policy	`Access-Control-Allow-Origin: *` in responses; unexpected Origin headers in access logs	Explicit origin allow-list; no wildcard on endpoints returning user data	cors middleware configuration; reverse-proxy rule; log the Origin header
3. Timestamp disclosure	Unix timestamps in response bodies	Review and remove timestamps clients do not need	Manual review; ZAP passive rule
4. SQL injection	SQL errors in logs; quote and UNION patterns in parameters; unusual query volume	Parameterised queries, server-side validation, least-privilege DB account, generic error messages	ORM bind parameters; SAST in CI; WAF as defence in depth; DB audit logging
5. Cross-site scripting	Script tags or event handlers in inputs; CSP violation reports	Context-aware output encoding; avoid innerHTML and bypassSecurityTrust on untrusted data; CSP; HttpOnly and Secure cookies	Angular sanitiser; CSP; SAST and lint rules
6. Broken authentication	Repeated failed logins; tokens with alg=none or odd claims	Strong random JWT secret, reject alg=none, short expiry, remove default credentials, MFA for admin	JWT library settings; rate limiter; SIEM alert on failed logins
7. Broken access control	Requests for other users' object IDs; spikes of 401/403 responses	Server-side authorisation and ownership check on every request; deny by default; unguessable IDs	Authorisation middleware; automated tests for object-level access
8. Sensitive data exposure	Requests to /ftp, backups or /metrics from outside	Remove public listings and backups; restrict /metrics; bcrypt or argon2 with salt; HTTPS and HSTS	Reverse-proxy ACLs; forced-browsing checks in ZAP
9–10. CSRF and brute force	State-changing requests with a foreign Origin or Referer; many failures per account or IP	Anti-CSRF tokens and SameSite cookies; rate limiting, lockout or back-off, CAPTCHA	Rate-limit middleware; CSRF token library; SIEM alerting

5.3 Risk Exposure Over Time

What happens if the three most serious risks stay open:

Risk 1 — SQL injection (Finding 4): a single injectable parameter lets an attacker read the whole database, including user data, payment information and password hashes, and automated tools make such flaws easy to find (TalkTalk, 2015: SQLMap located injectable fields and 156,959 customers' records were taken). The exposure compounds over time: stolen hashes enable credential stuffing against other sites, and a breach brings regulatory scrutiny and reputational damage.

Risk 2 — Missing CSP and permissive CORS combined with XSS (Findings 1, 2, 5): these two Medium gaps are cheap to fix now but weaken every other control. With no CSP any XSS flaw runs unconstrained, and with a wildcard CORS policy other sites can read unauthenticated responses. ZAP tagged both as systemic, so every page added to the application inherits them, and users' tokens stay one injected script away from theft.

Risk 3 — Broken authentication and access control (Findings 6, 7): weak token signing, guessable credentials and missing ownership checks allow account takeover and privilege escalation up to admin functions (A4). Facebook's 2019 contact-importer case (Phase A) shows how missing authorisation and rate limits on one API let attackers harvest data on hundreds of millions of users. Without logging and alerting (T12), the weakness can persist unnoticed and the exposure grows every day.

6.0 Recommendations

Each recommendation traces to a finding in Section 4. Priority follows the risk rating: Critical = Immediate, Medium and High = Short-term, Low = Long-term.

6.1 Technical Remediation

#	Finding	Specific Remediation Action	Priority	Standard / Reference
1	CSP header not set	Send a Content-Security-Policy header on every response, for example `default-src 'self'; script-src 'self'; object-src 'none'; base-uri 'self'; frame-ancestors 'self'`. Deploy it first as Content-Security-Policy-Report-Only, fix the violations, then enforce (Express: helmet.contentSecurityPolicy).	Short-term	OWASP CSP Cheat Sheet / CWE-693
2	Permissive CORS policy	Replace `Access-Control-Allow-Origin: *` with an explicit allow-list, for example `cors({ origin: ['https://shop.example.com'] })`, or remove CORS headers so the Same Origin Policy applies. Do not expose user-specific data on unauthenticated endpoints.	Short-term	A01:2025 / CWE-264
3	Unix timestamp disclosure	Confirm that 1666666667 is not sensitive (ZAP's recommendation) and remove timestamps that clients do not need.	Long-term	CWE-497
4	SQL injection	Use parameterised queries or ORM bind parameters for every query, validate input on the server, run the database account with least privilege and return generic error messages.	Immediate	OWASP SQL Injection Prevention Cheat Sheet / CWE-89
5	Cross-site scripting	Apply context-aware output encoding, avoid innerHTML and Angular bypassSecurityTrust on untrusted data, sanitise input, deploy the CSP from Finding 1 and set HttpOnly and Secure on cookies.	Immediate	OWASP XSS Prevention Cheat Sheet / CWE-79
6	Broken authentication	Sign JWTs with a strong random secret, reject alg=none, use short token expiry, remove default and guessable credentials and require MFA for admin accounts.	Immediate	OWASP Authentication Cheat Sheet / CWE-287
7	Broken access control	Enforce authorisation on the server for every request, check object ownership, deny by default and use unguessable identifiers.	Immediate	OWASP Authorization Cheat Sheet / CWE-284
8	Sensitive data exposure	Remove public file listings and backups (such as /ftp), restrict /metrics to internal access, hash passwords with bcrypt or argon2 plus salt, and enforce HTTPS with HSTS.	Immediate	OWASP Password Storage Cheat Sheet / CWE-200

#	Finding	Specific Remediation Action	Priority	Standard / Reference
9	Cross-site request forgery	Add anti-CSRF tokens, set SameSite=Lax or Strict on cookies, require the current password for sensitive changes and use POST or PUT, never GET, for state changes.	Short-term	OWASP CSRF Prevention Cheat Sheet / CWE-352
10	Brute force	Apply rate limiting, account lockout or exponential back-off and CAPTCHA, and alert on repeated failures.	Short-term	A07:2025 / CWE-307

6.2 Strategic & Operational Improvements

Beyond the individual fixes, these improvements would prevent similar findings from recurring:

- **Automate security checks in CI/CD:** run a ZAP baseline scan and a security-header check on every build, failing it when a CSP is missing or CORS is a wildcard; add SAST for injection and XSS patterns and dependency scanning for vulnerable components (T6; the Equifax lesson from Phase A).
- **Threat-model new features:** make threat modelling a standing step of the secure development lifecycle for critical components, as Phase A's Insecure Design category recommends, and repeat the Phase D exercise when the application changes.
- **Widen test coverage:** the Phase C scan made only GET requests. Add authenticated scans of POST and PUT endpoints and manual testing of the Critical threats (T1–T5, T10) to confirm or refute them.
- **Detect and respond:** centralise logging and alerting for failed logins, access-control failures and input-validation failures (T12), and define an incident-response plan.
- **Harden deployment defaults:** manage headers, CORS, cookie flags and secrets as configuration under version control, and bind intentionally vulnerable lab targets to 127.0.0.1 (Phase B observation).

7.0 Reflection

Reflections on the assessment, grounded in the findings above.

Q1 Which finding presents the greatest risk, and why?

SQL injection (Finding 4, score 25). It combines the highest likelihood and impact: one injectable input can expose every user record and password hash in the database, and tools like SQLMap automate the discovery, as TalkTalk's 2015 breach showed. I should be clear that this is a modelled threat in Phase D, not an exploit I demonstrated.

Q2 How did the scan results compare with the threat model?

They complement each other. ZAP confirmed three real configuration gaps (no CSP, wildcard CORS, a disclosed timestamp) but found nothing in the Critical band, because the alerts came from passive checks plus one active rule and every request was a GET. The threat model reached the more serious risks (injection, authentication, access control) by reasoning about assets and attack paths. The two meet at XSS: the missing CSP makes the modelled XSS threat worse.

Q3 How would you explain the key risks to a non-technical stakeholder?

The shop has no automatic checks stopping attackers from tampering with what its web pages do, and the way it is built could let a stranger ask the database for customers' details or act as another user. Two easy fixes, adding a browser safety setting and restricting which sites may read our data, reduce the damage if something slips through. The more serious weaknesses need developer time to fix and a proper test to confirm. Left alone, they put customer data, trust and regulatory standing at risk.

Q4 What would you do differently or next?

I would run an authenticated, active ZAP scan covering POST and PUT endpoints, then test the Critical threats by hand to confirm them. I would also add the ZAP baseline and header check to a CI pipeline, and start the lab container bound to 127.0.0.1 rather than all interfaces.

8.0 Deliverables Submission Checklist

Deliverables for each phase, with the evidence filenames used in this report.

✔	Phase	Deliverable Description	Your Filename	Status
☑️	Phase A	OWASP Top 10 summary with real-world breach case studies	PhaseA_OWASP_Top10_Summary.md	Complete
☑️	Phase B	Lab setup screenshot: Juice Shop container running on Kali Linux (Figure 1)	PhaseB_Docker_JuiceShop.png	Complete
☑️	Phase C	ZAP scan report for http://localhost:3000	PhaseC_ZAP_Report.html	Complete
☑️	Phase D	Threat model: assets, threats, risk matrix and mitigations	PhaseD_ThreatModel.md	Complete
☑️	Phase E	TryHackMe completed-rooms screenshot (Figure 5)	PhaseE_TryHackMe_Profile.png	Complete
☑️	Phase F	This report, exported as PDF	ASE_APPSEC_Report.pdf	Complete

Submission naming:

- Report PDF: ASE_APPSEC_Report.pdf (name given in the Phase F brief)
- Evidence files: PhaseA_OWASP_Top10_Summary.md, PhaseB_Docker_JuiceShop.png, PhaseC_ZAP_Report.html, PhaseD_ThreatModel.md, PhaseE_TryHackMe_Profile.png

9.0 Additional Observations

Additional observations from the task:

- **Tool behaviour:** ZAP reported 91% memory use, 2 errors and 11 warnings during the scan, and still produced the report. Docker published the Juice Shop container on all interfaces (0.0.0.0:3000); `-p 127.0.0.1:3000:3000` would limit it to the lab machine.
- **Connections:** the missing CSP (ZAP) links to the XSS threat (T3), and the wildcard CORS policy to broken access control (T4). The Phase A case studies show the same controls recurring: patching, least privilege and logging.
- **Scoring notes:** Findings 1–3 were scored in this report because Phase D did not cover them. CSRF (T10, score 16) was labelled High in Phase D; this report uses the template's score guide, which makes it Critical.

Declaration & Sign-Off

I confirm that:

- All work in this report is my own except where AI usage is explicitly declared in Section 2.3.
- All technical activities were performed exclusively within the authorised environments listed in Section 1.3.
- No external systems, production networks, or third-party infrastructure were accessed during this task.
- I have reviewed this report for accuracy, completeness, and professional presentation before submission.

Intern Full Name	Signature	Date of Submission
Abdoul Karim Savadogo		12 October 2026

EncryptEdge Labs Limited — Junior Application Security Engineer Internship Programme
Company No. 15830711 | United Kingdom | encryptedgelabs.com

Copyright © 2026 EncryptEdge Labs. All rights reserved. | Report Template v3.0